

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL – 576104
DEPARTMENT OF HUMANITIES AND MANAGEMENT

CERTIFICATE

This is to certify that Ms./Mr. Reg.
No..... Section: Roll No.....has satisfactorily
completed the lab exercises prescribed for Banking Technology Lab [**CSF 3142**] of Second Year
B. Tech. Degree at MIT, Manipal, in the academic year 2026-2027.

Date:

Signature

Faculty in Charge

Signature

Head of the Department

LAB NO.	TITLE	PAGE NO.	REMARKS
	Course Objectives and Outcomes		
	Evaluation plan	ii	
	Instructions to the Students	iii	
L1	Login Page --> API for user registration		
L2	Login Page --> User authentication, Password masking		
L3	Login page --> User authorization, integrate with customer module built in Fintech Lab		
L4			
L5	Overall Solution document for 2 modules (Product and Pricing, Login), API signatures - Sample inputs and outputs		
L6	Overall Solution document for 2 modules (FD Calculator and FD Module), API signatures - Sample inputs and outputs		
L7	Presentation and Execution of Basic login process, login page with Customer module integrated		
L8			
L9			
L10	Presentation and Execution of Product and Pricing, login page with Product and Pricing integrated		
L11			
L12			
L13			
L14	Presentation and Execution of FD Calculator, login page with FD Calculator integrated		
L15			

L16	Execution of Account creation and initial transaction		
L17			
L18			
L19	Execution of Batch scheduler and Batch processes		
L20			
L21	Presentation and Execution of FD Module, login page with FD Module integrated		
L22			
L23	Presentation and Execution of Report, login page with Report Module integrated		
L24	Overall Presentation and Execution		
L25	Overall Presentation and Execution		

1. Introduction

Banking Technology Lab offers an opportunity to apply software engineering practices in the context of real-world banking systems. Through this lab, you will design, develop, and deploy modular applications that simulate core banking functionalities, ranging from product definition and customer onboarding to fixed deposit account handling and operational reporting.

All modules you develop will be integrated into a single, cloud-hosted banking application deployed using Azure App Service and container technologies. The focus will be on clean architecture, security, integration, and real-time communication across components.

2. Lab Objectives

- To design and develop modular applications simulating banking operations.
- To compose independently built services into an integrated banking platform.
- To implement and test key banking processes including account creation, interest computation, maturity handling, and customer reporting.
- To apply security measures such as password hashing, authentication tokens, and PII data protection.
- To deploy applications in cloud environments using Docker, Kubernetes, and Azure App Service.
- To implement and evaluate batch jobs, event publishing, and readiness/liveliness probes.
- To explore the architecture, business rules, and communication channels used in real-world banking systems.

3. Technology Stack

Category	Technologies/Tools
Frontend	Angular, HTML5, CSS3
Backend	Java with Spring Boot (REST APIs)
Database	MySQL
Report Generation	Python (for generating operational and customer reports)
Containerization	Docker, Kubernetes

Cloud Deployment	Azure App Service
Security	JWT Tokens, Bcrypt (for password hashing and verification)
Development Tools	Dev Containers (VS Code), Git, Postman
Monitoring	Readiness & Liveness Probes, Elastic Scaling
Other Integrations	Email/SMS Alerts, Event Publishing, File Extracts

4. Learning Outcomes

By the end of the lab, you will be able to:

- Develop and integrate banking modules using full-stack technologies.
- Distinguish between administrative functions and end-user workflows in a banking application.
- Simulate and compare fixed deposit interest calculations using simple and compound models.
- Manage account life cycles including creation, interest computation, payouts, premature closure, and maturity processing.
- Build and test batch operations like interest accrual, statement generation, and renewal automation.
- Apply debit/credit operations with correct balance classifications and GL mappings.
- Implement secure login, authentication, and encrypted data handling.
- Publish and consume events, perform file extracts, and implement compliance reporting.
- Apply caching, API versioning, and container scaling principles for robust deployment.

Instructions for the Banking Technology Lab

Pre-Lab Session Guidelines

1. Ensure you carry your Banking Technology Lab Manual and required stationery to each session.
2. Arrive on time and adhere to the institution's dress code.
3. Sign the Lab Attendance Register promptly upon arrival.
4. Sit at your assigned system and confirm your attendance as instructed.
5. Maintain lab discipline and always observe professional behavior.
6. Review the assigned module or exercise beforehand to come prepared for implementation and discussion.

During the Lab Session

1. Follow the instructions provided for each assigned lab activity precisely.
2. Upon completion of your exercise, demonstrate the working code and output to the faculty-in-charge for verification.
3. After approval, neatly copy the program, inputs/outputs, and observations into your lab record book.
4. Keep prescribed textbooks, lab manuals, or reference notes accessible for quick consultation when necessary.

General Instructions for Lab Exercises

1. All exercises must be completed individually. Group coding or code sharing is strictly prohibited.
2. Your lab observation record must include:
 - Clear and structured code
 - Input and output data

- Evidence of API responses or system behavior (where applicable)
- 3. Plagiarism, code duplication, or copying from others will result in academic penalties.
- 4. Exercises are categorized into:
 - Solved Examples – reference material explained during lab
 - Assigned Lab Tasks – to be completed during the session
 - Additional Exercises – for practice outside lab hours
- 5. If a session is missed, obtain faculty approval for a repeat session. Note: Credit is awarded only for the original scheduled lab unless permitted by exception.
- 6. Lab tests and examinations may include new or modified versions of exercises beyond those listed in the manual.

Prohibited Activities

1. Do not bring mobile phones or personal electronic gadgets into the lab unless explicitly authorized.
2. Do not leave the lab without permission from the faculty or lab instructor.

Evaluation Plan (Total: 100 Marks)

- **Continuous Assessment:** 85 marks
- **Lab Viva:** 15 marks

Module-wise Assessment (85 Marks)

- Login & UI Integration – 10 marks
- User Auth, Hashing, Authorization – 5 marks
- Customer Module (PII Masking, Setup) – 5 marks
- Product & Pricing – 10 marks
- FD Calculator (Simple & Compound) – 10 marks
- Fixed Deposit Module – 35 marks
 - Includes account creation, transactions, batch processing, maturity, alerts, file extract, scaling, and cloud deployment
- Reports Module – 5 marks
- Documentation & Final Presentation – 5 marks

Lab L1: Implementation of Login API for User Registration

1. Objective

To design and implement a secure, role-ready user registration system using Spring Boot and MySQL. This module lays the foundation for user identity management in the banking application, ensuring encrypted credential storage and readiness for future authentication and authorization flows.

2. Technical Overview

User registration is the first step in any secure web application. In a banking technology system, where user accounts control access to sensitive customer data, credentials must be handled with strong encryption and data validation policies.

This lab focuses on:

- Accepting a new user's details (username, password, email)
- Encrypting the password using Bcrypt with salt
- Storing credentials in a relational database (MySQL)
- Ensuring username uniqueness to prevent collision
- Structuring the backend API following RESTful principles

Once implemented, this API becomes the entry point for all users into the system.

3. System Architecture and Workflow

```
[Angular UI]
↓
POST /api/register
↓
[Spring Boot REST API]
↓
Encrypt password using Bcrypt
↓
Store into MySQL if username is unique
↓
Return success or error response
```

4. Database Design (MySQL)

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    email VARCHAR(100),  
    role VARCHAR(20) DEFAULT 'CUSTOMER',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5. Spring Boot Implementation

5.1 Dependencies (in pom.xml)

xml

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-security</artifactId>  
</dependency>  
  
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>  
  
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>mysql</groupId>
```

```
    <artifactId>mysql-connector-java</artifactId>
```

```
</dependency>
```

5.2 Entity Class

```
@Entity
```

```
@Table(name = "users")
```

```
public class User {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String username;
```

```
    private String passwordHash;
```

```
    private String email;
```

```
    private String role = "CUSTOMER";
```

```
}
```

5.3 Repository Interface

```
public interface UserRepository extends JpaRepository<User, Long> {
```

```
    Optional<User> findByUsername(String username);
```

```
}
```

5.4 Controller

```
java
```

```

@RestController

@RequestMapping("/api")

public class AuthController {

    @Autowired

    private UserRepository userRepo;

    @Autowired

    private PasswordEncoder passwordEncoder;

    @PostMapping("/register")

    public ResponseEntity<?> registerUser(@RequestBody User user) {

        if (userRepo.findByUsername(user.getUsername()).isPresent()) {

            return ResponseEntity.status(HttpStatus.CONFLICT)

                .body(Map.of("status", "error", "message", "Username already exists"));

        }

        user.setPasswordHash(passwordEncoder.encode(user.getPasswordHash()));

        userRepo.save(user);

        return ResponseEntity.ok(Map.of("status", "success", "message", "User registered successfully"));

    }

}

```

5.5 Configuration

java

```

@Configuration

public class AppConfig {

    @Bean

```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}  
}
```

6. Angular Frontend (Brief)

- Simple registration form with input fields for username, password, and email
- Angular sends data via HTTP POST to /api/register

7. Expected Output

- Successful registration returns:

json

```
{ "status": "success", "message": "User registered successfully" }
```

- Duplicate registration returns:

json

```
{ "status": "error", "message": "Username already exists" }
```

Lab L2 – User Authentication and Password Masking

1. Objective

To implement user login (authentication) functionality using Spring Boot and MySQL and enforce password masking at the front end using Angular. This lab ensures users can securely access the system using encrypted credentials.

2. Technical Overview

Authentication checks if the entered credentials match what's stored in the system. In this lab, once a user submits a login request:

- The username is verified against the database
- The input password is matched against the stored hash using Bcrypt
- If valid, a JWT token may be generated for session tracking
- If invalid, the system returns an appropriate error

On the UI, the password input field should be masked, preventing it from being visible while typing.

3. Workflow Diagram

```
[Angular UI]
↓
POST /api/login with masked password input
↓
[Spring Boot Backend]
↓
Fetch user by username from MySQL
↓
Validate input password with stored hash (Bcrypt)
↓
Return login status (and JWT if enabled)
```

4. Spring Boot Implementation

4.1 Login Controller

```
java
```

```

@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest login) {
    Optional<User> userOpt = userRepo.findByUsername(login.getUsername());

    if (userOpt.isEmpty()) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
username or password");
    }

    User user = userOpt.get();
    if (passwordEncoder.matches(login.getPassword(), user.getPasswordHash()))
    {
        return ResponseEntity.ok(Map.of("status", "success", "message",
"Login successful"));
    } else {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
credentials");
    }
}

```

4.2 DTO Class

```

java
public class LoginRequest {
    private String username;
    private String password;
}

```

5. Angular Frontend (UI Highlights)

```

html
<input type="password" [(ngModel)]="password" placeholder="Enter Password" />

```

- Use Angular's `HttpClient` to POST login details securely
- Display messages on success/failure
- Store JWT token (optional) in local storage

6. Technologies Used

Component	Technology
Backend	Java (Spring Boot)
Database	MySQL
UI	Angular
Security	Bcrypt
API Testing	Postman

7. Expected Output

- Valid login returns "Login successful"
- Invalid credentials are rejected securely
- Password field is masked on UI input

Lab L3 – User Authorization and Integration with the Customer Module

1. Objective

To implement user authorization after authentication and integrate it with the Customer Module developed earlier. This lab builds on Lab L2 by enabling role-based access control (RBAC) and ensuring that only authorized users can access specific modules, such as Customer onboarding or profile management.

2. Technical Overview

Authorization comes after successful authentication. While authentication verifies who the user is, authorization defines what the user is allowed to do. In a banking context, access to various modules, such as customer onboarding, FD creation, or reporting, should be strictly controlled based on roles such as:

- **ADMIN**
- **BANK_OFFICER**
- **CUSTOMER**

This lab covers:

- Adding role assignment during registration or by an admin
- Enforcing access permissions using Spring Security
- Integrating the authorization logic with the existing Customer Module
- Securing the Customer API endpoints so only authorized users can access them

3. System Flow & Architecture

```
[Angular UI]
|
POST /login
↓
[Spring Boot Auth Controller] --(Authentication)
|
Generate JWT Token
↓
[Angular Frontend Stores Token]
```

↓
Frontend Calls /customer/profile with Token
↓
Spring Security Filter Validates Token + Role
↓
Allows or Denies Access Based on Role

4. Implementation Details

4.1 Database Schema – Add Role Field

Extend the users table with a role column:

```
sql
ALTER TABLE users ADD COLUMN role VARCHAR(20) DEFAULT 'CUSTOMER';
```

Example roles:

- 'CUSTOMER'
- 'BANK_OFFICER'
- 'ADMIN'

4.2 Spring Boot – Security Configuration

```
java
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private UserDetailsServiceImpl userDetailsService;

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .authorizeRequests()
            .antMatchers("/api/auth/**").permitAll()
            .antMatchers("/api/customer/**").hasAnyRole("CUSTOMER",
"BANK_OFFICER", "ADMIN")
            .anyRequest().authenticated()
            .and()

        .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS);

        http.addFilterBefore(jwtFilter(),
UsernamePasswordAuthenticationFilter.class);
    }

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
```

```

        auth.userDetailsService(userDetailsService).passwordEncoder(new
BCryptPasswordEncoder());
    }

    @Bean
    public JwtAuthenticationFilter jwtFilter() {
        return new JwtAuthenticationFilter();
    }
}

```

4.3 Spring Boot – User Entity

```

java
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;

    private String password;

    private String role; // CUSTOMER, ADMIN, etc.

    // Getters and setters...
}

```

4.4 JWT Token Generation (Upon Successful Login)

```

Java

public String generateToken(UserDetails userDetails) {
    return Jwts.builder()
        .setSubject(userDetails.getUsername())
        .claim("role",
userDetails.getAuthorities().iterator().next().getAuthority())
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 *
60 * 10))
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .compact();
}

```

The token will now contain the role claim, which is decoded by the frontend and backend during request validation.

4.5 Angular – Attaching JWT Token in Headers

```

typescript
const headers = new HttpHeaders({
  'Authorization': `Bearer ${token}`
});

```

```
this.http.get('/api/customer/profile', { headers }).subscribe(...);
```

4.6 Securing the Customer Module

Update the controller to enforce authorization using annotations:

```
java
@RestController
@RequestMapping("/api/customer")
public class CustomerController {

    @GetMapping("/profile")
    @PreAuthorize("hasAnyRole('CUSTOMER', 'BANK_OFFICER')")
    public ResponseEntity<Customer> getCustomerProfile(Authentication auth) {
        String username = auth.getName();
        Customer customer = customerService.getProfileByUsername(username);
        return ResponseEntity.ok(customer);
    }
}
```

5. Integration with Customer Module

- After logging in, a user with the `CUSTOMER` role will be allowed to access only the `/api/customer/**` endpoints.
- All requests to these endpoints must include a valid JWT token.
- Angular frontend must decode the token to control which menu items or screens are visible based on role.

6. Security Considerations

- Do not hardcode roles on the client side; enforce them on the server
- Ensure the JWT token is stored securely in memory or local storage
- Set token expiration and allow logout by simply removing the token
- Invalidate tokens if the user is disabled or deleted (advanced)

7. Technologies Used

Component	Stack
Frontend	Angular
Backend	Java (Spring Boot)
Database	MySQL

8. Expected Output

- Unauthorized users are blocked from accessing `/api/customer/**`
- Only users with appropriate roles are allowed to use the module
- The JWT token is correctly generated and validated
- Frontend hides or shows UI elements based on user role

Lab L4 – Advanced User Authorization and Role-Based Access Integration with Customer Module

1. Objective

To extend the authorization mechanism implemented in Lab L3 by integrating deeper **role-based access control (RBAC)** into the **Customer Module**. This includes refining how user roles (ADMIN, BANK_OFFICER, CUSTOMER) interact with specific resources and enforcing authorization directly at the API, service, and UI levels.

2. Technical Overview

In modern banking systems, every functional module must be protected from unauthorized access. While Lab L3 introduced basic role-based control, this lab focuses on refining it for production readiness:

- **API-level authorization enforcement** using Spring Security annotations
- **Service-layer filtering** to ensure business logic respects user permissions
- **Frontend routing restrictions** to dynamically show/hide UI elements based on roles
- Integration with the existing **Customer Module**, which includes functions such as profile management, FD account linking, and customer status

This lab also introduces **method-level security annotations**, which provide an additional safety layer even if APIs are accidentally exposed.

3. System Flow and Architecture

```
plaintext
[Angular UI]
  ↓ (Sends JWT Token)
[API Gateway or Angular HTTP Client]
  ↓
[Spring Boot Backend with SecurityConfig]
  ↓
[Customer Controller]
```

↓
[Customer Service - Role Filter]
↓
[MySQL Database Access via JPA]

4. Database Structure and Sample Data

Ensure the following roles are clearly mapped in the `users` table.

MySQL Table: `users`

Field	Type	Sample Data
<code>id</code>	INT	101
<code>username</code>	VARCHAR(50)	"admin1"
<code>password</code>	VARCHAR(255)	[Bcrypt Hash]
<code>email</code>	VARCHAR(100)	"admin@bank.com"
<code>role</code>	VARCHAR(20)	ADMIN
<code>created_at</code>	TIMESTAMP	CURRENT_TIMESTAMP

5. Backend Implementation (Spring Boot)

5.1 Enable Global Method Security

```
java
@EnableGlobalMethodSecurity(prePostEnabled = true)
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    // ... as in Lab L3
}
```

5.2 Controller (CustomerController.java)

```
java
@RestController
@RequestMapping("/api/customer")
```

```

public class CustomerController {

    @Autowired
    private CustomerService customerService;

    @PreAuthorize("hasRole('BANK_OFFICER') or hasRole('ADMIN')")
    @GetMapping("/all")
    public ResponseEntity<List<Customer>> getAllCustomers() {
        return ResponseEntity.ok(customerService.getAllCustomers());
    }

    @PreAuthorize("hasRole('CUSTOMER')")
    @GetMapping("/profile")
    public ResponseEntity<Customer> getOwnProfile(Authentication
authentication) {
        String username = authentication.getName();
        return
ResponseEntity.ok(customerService.getCustomerProfile(username));
    }
}

```

5.3 Service Layer (CustomerService.java)

```

java
public Customer getCustomerProfile(String username) {
    Optional<User> user = userRepository.findByUsername(username);
    if (user.isPresent()) {
        return userRepository.findByUser(user.get());
    } else {
        throw new RuntimeException("User not found");
    }
}

```

6. Frontend (Angular) Authorization Integration

6.1 Store Role in JWT Payload

Ensure the backend generates a JWT token with the `role` embedded:

```
json
{
  "sub": "admin1",
  "role": "ADMIN",
  "exp": 1683763440
}
```

6.2 Decode Token in Angular

```
typescript
const decoded = jwtDecode(token);
this.role = decoded['role'];
```

6.3 Conditionally Render Routes/Components

```
html
<div *ngIf="role === 'BANK_OFFICER' || role === 'ADMIN'">
  <a routerLink="/customer/all">View All Customers</a>
</div>
```

7. Security Best Practices

- Method-level annotations ensure defense-in-depth
- API must always reject unauthorized access even if frontend UI is bypassed
- Token expiration must be enforced and auto logout handled
- Frontend must not rely solely on UI restriction; always verify at backend

8. Technologies Used

Layer	Stack
Frontend	Angular
Backend	Spring Boot + Spring Security
Database	MySQL
Security	Bcrypt, JWT, @PreAuthorize

Testing Tools	Postman, Swagger
---------------	------------------

9. Expected Output

- **/api/customer/all** is accessible only to BANK_OFFICER or ADMIN
- **/api/customer/profile** is accessible only to the logged-in CUSTOMER
- Unauthorized access returns 403 Forbidden
- Angular UI adjusts visibility based on the decoded role

Lab L5 – Product and Pricing Module: API Development and Documentation

1. Objective

To develop and document RESTful APIs for the **Product and Pricing Module**, enabling the creation, maintenance, and querying of banking products (such as FDs, savings accounts, etc.). The lab also emphasizes generating solution documentation including API signatures, sample inputs/outputs, and use-case validations.

2. Technical Overview

The Product and Pricing Module is a central component that defines banking products. All accounts are derived from these products, which encapsulate business rules like minimum amounts, term durations, interest configurations, and associated charges.

This lab focuses on:

- Creating and updating product definitions via APIs
- Retrieving product details based on filters
- Structuring solution documents to include API specs, parameters, and expected behavior

3. Product Structure

Each product includes:

- **Basic Info:** Name, code, type, currency, effective date
- **Business Rules:** Min/max terms, rate limits, transaction types
- **Charges:** Interest credit, tax, withdrawal fees
- **Roles:** Owner, co-owner, nominee, etc.

4. API Specifications

4.1 Create Product (POST)

- **Endpoint:** `/api/product`
- **Payload:**

```
json
{
  "productCode": "FD001",
  "productName": "Fixed Deposit - Short Term",
  "productType": "FD",
  "currency": "INR",
  "effectiveDate": "2025-07-01",
  "termLimit": { "min": 3, "max": 36 },
  "rateLimit": { "min": 5.0, "max": 7.5 }
}
```

4.2 Get Product (GET with filter)

- **Endpoint:** /api/product/search
- **Query Params:** productType=FD&startDate=2025-01-01&endDate=2025-12-31

4.3 Maintain Product (PUT)

- Update product name or interest range

5. Expected Output

- Product stored with all metadata and retrievable via queries
- API documentation includes sample inputs and outputs
- Functional integration with FD and Calculator modules in later labs

Lab L6 – Fixed Deposit Calculator: API and Reporting Integration

1. Objective

To develop a **Fixed Deposit Calculator** that computes maturity amount based on user inputs and interest matrix. Also, generate report outputs and integrate with Product and Pricing module for fetching base rates.

2. Technical Overview

The FD Calculator must support:

- **Simple Interest:** $M = P + (P \times r \times t / 100)$
- **Compound Interest:** $M = P \times (1 + r/n)^{nt}$

Rates vary based on **customer category** (e.g., Senior Citizen +1%). The logic must restrict combined additional percentage to not exceed configured cap (e.g., 2%).

3. API Specification

3.1 Calculate FD

- **Endpoint:** `/api/fd/calculate`
- **Payload:**

```
json
{
  "principal": 100000,
  "term": { "value": 12, "unit": "months" },
  "baseRate": 6.5,
  "categories": ["Senior Citizen", "Employee"]
}
```

- **Output:**

```
json
{
  "maturityAmount": 107000,
  "effectiveRate": 7.0,
  "interestEarned": 7000
}
```

4. UI Integration

- Dropdown for categories (fetched from rate matrix)
- Auto-display of additional % per selected category
- Enforce cap logic before submission

5. Expected Output

- Valid maturity calculations for FD products
- Dynamic rate merging based on categories
- Display and export of result reports (Python-based reporting)

Lab L7 – Execution of Login Process with Customer Module Integration

1. Objective

To demonstrate the working login system and validate integration with the Customer module. This includes session validation, role-based routing, and secure communication of user context.

2. Technical Workflow

1. User logs in → Token returned
2. Token stored in frontend → Attached to all module requests
3. Customer Module reads token → Extracts `username` and `role`
4. Filters customer data based on token holder

3. Key Features

- Token-based login
- Masked password on frontend
- Customer module accepts JWT and uses `Authentication` object
- Routes differ for `CUSTOMER` and `ADMIN/BANK_OFFICER`

4. Validation

- Login with customer user → Can only view own profile
- Login with officer/admin → Can view all profiles

Lab L8 – Early Integration and Module Expansion Testing

1. Objective

To perform early integration testing across Login, Product, Calculator, and Customer modules. This lab aims to ensure APIs are correctly wired, tokens are propagated, and business logic boundaries are enforced.

2. Activities

- Run full flow: Login → Create Product → Calculate FD → Access Customer
- Test invalid access scenarios (e.g., missing token, unauthorized role)
- Begin documenting module interdependencies
- Log events (account creation, product definition)

3. Tools for Testing

- **Postman collections** for all endpoints
- **Angular debugging** via Developer Tools
- Logback or SLF4J logging on backend

4. Outcome

- Verified that tokens flow across modules
- Functional flow from login to account-linked calculator tested
- Initial system diagram can now be created for documentation

Lab L9 – Integration of Product and Pricing Module with Login System

1. Objective

To perform secure integration of the **Product and Pricing Module** with the login and authorization framework. Only authenticated and authorized users (e.g., `BANK_OFFICER`, `ADMIN`) should be able to create, update, or query product information.

2. Technical Overview

This lab ensures that product management features are protected and accessible only to users with valid JWT tokens. The focus is on:

- Token propagation from login
- Authorization filtering in product-related APIs
- Secure session enforcement between modules

3. Backend Enhancements (Spring Boot)

- Add `@PreAuthorize("hasAnyRole('BANK_OFFICER','ADMIN')")` to all sensitive product endpoints
- Inject user context using `Authentication` or custom `SecurityContext`

```
java
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/product")
public ResponseEntity<?> createProduct(@RequestBody ProductDTO dto,
Authentication auth) {
    String user = auth.getName(); // Audit tracking
    return productService.save(dto, user);
}
```

4. Frontend Handling (Angular)

- Use `HttpInterceptor` to auto-attach JWT token in all requests

- Disable product creation/update UI options for unauthorized users

5. Outcome

- Verified that product management APIs reject unauthenticated or unauthorized users
- Product creation requires JWT and role-based validation

Lab L10 – Integration of Fixed Deposit Calculator with Login Module

1. Objective

To integrate the FD Calculator with the login system to ensure:

- Only authenticated users can simulate maturity estimates
- The UI dynamically adjusts interest parameters based on logged-in customer category

2. Key Features

- Calculator fetches logged-in user's category (e.g., EMPLOYEE, SENIOR)
- Additional interest benefits are applied dynamically
- Access restricted to CUSTOMER role

3. API Enhancement

```
java
@PreAuthorize("hasRole('CUSTOMER')")
@PostMapping("/fd/calculate")
public ResponseEntity<?> calculateFd(@RequestBody FDInput input, Authentication
auth) {
    String username = auth.getName();
    List<String> categories = customerService.getCategories(username);
    return calculatorService.calculate(input, categories);
}
```

4. Output

- Calculator returns personalized maturity output
- Unauthorized users receive HTTP 403
- Report previews can be generated per calculation

Lab L11 – FD Calculator Testing and Enhancements

1. Objective

To validate the FD calculator logic against multiple scenarios and enhance its logic for:

- Rate limits enforcement
- Combined category caps
- Report generation through Python script (executed via Spring Boot)

2. Testing Scenarios

Category Set	Base Rate	Add-on	Final Rate	Capped?
EMPLOYEE + SENIOR	6.5%	+2.0%	8.5%	Yes
EMPLOYEE + PREMIUM_USER	6.5%	+3.0%	8.5%	Yes

Implement backend enforcement to cap the effective rate based on product-defined maximums.

3. Report Output (via Python)

```
csv
Username,Principal,Term,BaseRate,CategoryAddOn,FinalRate,Maturity
alice123,100000,12M,6.5,2.0,8.5,108500
```

Python script (triggered via shell call from Java):

```
java
Runtime.getRuntime().exec("python3 generate_fd_report.py");
```

4. Outcome

- Logic validated for compound scenarios
- Report generation integrated and triggered
- System ready for FD module to consume this logic

Lab L12 – Setup for FD Module Implementation

1. Objective

To initialize and prepare all required infrastructure and services for the **Fixed Deposit Module**, including account number generation plugin, FD account structure, and transaction baseline.

2. Tasks in This Lab

- Define `fd_accounts` table in MySQL with fields:
 - `fd_account_no`, `product_code`, `customer_id`, `term`, `rate`, `amount`, `status`, `created_at`, etc.
- Implement account number generation logic:
 - Pattern: [3-digit branch][6-digit seq][1-digit check]
- Build a pluggable account number service:

```
java
public interface AccountNumberGenerator {
    String generate(String branchCode);
}
```

3. Entity Design (Spring Boot)

```
java
@Entity
public class FixedDepositAccount {
    @Id
    private String fdAccountNo;

    private String customerId;
    private String productCode;
    private Double principal;
    private Double rate;
    private Integer term; // in months
    private String status; // ACTIVE, CLOSED
}
```

4. Integration Points

- Product code must be validated against Product module
- Rate must not exceed limits
- Term must match product's configured boundaries

5. Security and Role Control

- Only BANK_OFFICER can create new FD accounts
- Customer can view their own FD records only

6. Expected Outcome

- Complete FD account model with validation logic
- Foundation laid for creating, transacting, and reporting on FD accounts (to be done in Labs 13–16)

Lab L13 – Fixed Deposit Account Creation and Validation Process

1. Objective

To implement the process of Fixed Deposit (FD) account creation, integrating the FD Module with the Product and Pricing Module. This lab focuses on the automation of account number generation, validation of product configurations, and secure creation of customer-linked FD accounts.

2. Technical Overview

Fixed Deposit account creation is a core banking process where a customer selects an FD product and opens a deposit account. The module must ensure:

- The selected product exists and is active.
- All FD terms (interest rates, term limits, minimum deposit amounts) conform to the product configuration.
- The account number is generated programmatically and follows the defined structure.

This account creation process will enforce authorization rules, where only BANK_OFFICER and ADMIN roles are allowed to create FD accounts.

3. Workflow Architecture

```
plaintext
[Angular UI] - Account Creation Form
    ↓
POST /api/fd/account/create
    ↓
[Spring Boot FD Controller]
    ↓
Validate Product Code & Term against Product Module
    ↓
Generate Unique FD Account Number
    ↓
Persist FD Account in MySQL
    ↓
Return Account Details
```

4. FD Account Structure

Field	Description
fdAccountNo	Auto-generated, unique identifier
customerId	Foreign key to the user or customer table
productCode	Links to Product module
principalAmount	Initial deposit amount
interestRate	Final rate after category adjustments
termMonths	FD duration
status	ACTIVE / CLOSED
createdAt	Account opening date

5. Backend Implementation (Spring Boot)

5.1 Account Number Generation Service

```
java
public class FDAccountNumberGenerator {
    public String generate(String branchCode) {
        int seq = generateSequence(); // Example: from a DB sequence
        return branchCode + String.format("%06d", seq) +
            calculateChecksum(branchCode, seq);
    }

    private int generateSequence() {
        // Could use MySQL auto_increment or a dedicated sequence table
        return new Random().nextInt(999999);
    }

    private String calculateChecksum(String branch, int seq) {
        int sum = branch.hashCode() + seq;
        return String.valueOf(sum % 9); // Simple checksum logic
    }
}
```

5.2 FD Account Creation API

```
java
@PreAuthorize("hasRole('BANK_OFFICER')")
@PostMapping("/fd/account/create")
public ResponseEntity<?> createFDAccount(@RequestBody FDAccountRequest
request, Authentication auth) {
    Product product =
productService.validateProduct(request.getProductCode(),
request.getTermMonths());
}
```

```

        double rate = rateService.determineFinalRate(request.getProductCode(),
request.getCustomerId());
        String fdAccountNo =
accountNumberGenerator.generate(request.getBranchCode());

        FixedDepositAccount fd = new FixedDepositAccount(fdAccountNo,
request.getCustomerId(),
            request.getProductCode(), request.getPrincipalAmount(), rate,
request.getTermMonths());

        fdAccountRepository.save(fd);
        return ResponseEntity.ok(Map.of("fdAccountNo", fdAccountNo, "status",
"Account Created"));
    }

```

6. Database Design (MySQL)

Table: fd_accounts

Field	Type
fd_account_no	VARCHAR(20) PK
customer_id	VARCHAR(20)
product_code	VARCHAR(10)
principal	DOUBLE
interest_rate	DOUBLE
term_months	INT
status	VARCHAR(20)
created_at	TIMESTAMP

7. Frontend Implementation (Angular)

- Create **FD Account Creation Form**
- Dropdown to select:
 - Product Code (from Product module)
 - Term in months
 - Principal amount
- Auto-fill or fetch:

- Available interest rates
- User's category adjustments

8. Expected Output

- FD account is created with a unique account number.
- Valid product and term data are enforced.
- Customer receives their new FD account number in the response.
- Unauthorized roles are blocked with `403 Forbidden`.

Lab L14 – Interest Calculation, Capitalization, and Payout Process for Fixed Deposit Accounts

1. Objective

To implement the interest calculation engine for Fixed Deposit accounts, handle the capitalization of interest, and manage the interest payout or reinvestment process. This lab covers the business logic behind how banks accrue and apply interest to FD accounts, including scheduled batch processing.

2. Technical Overview

Interest in Fixed Deposit accounts can be capitalized (added to the principal) or paid out periodically (monthly, quarterly, yearly). The calculation can be based on:

- Simple interest
- Compound interest
- Custom payout schedules

This process typically runs as a batch job, calculating interest for all active FD accounts.

3. System Workflow

```
plaintext
[Batch Job or Scheduled Task]
    ↓
Retrieve all ACTIVE FD accounts
    ↓
For each account:
    ↓
Apply interest formula based on product settings
    ↓
Either:
    - Add to principal (capitalization)
    - Queue for payout (credit to customer account)
    ↓
Update account record and generate transaction record
```

4. Interest Calculation Formulae

4.1 Simple Interest:

```
plaintext
Interest = (Principal * Rate * TermMonths) / (12 * 100)
```

4.2 Compound Interest (Monthly Compounding):

```
plaintext
MaturityAmount = Principal * (1 + (Rate / (12 * 100))) ^ TermMonths
Interest = MaturityAmount - Principal
```

The **product's interest compounding type** determines which formula applies.

5. Spring Boot Batch Implementation

5.1 Batch Scheduler Setup

```
java
@Component
public class InterestCalculationScheduler {

    @Scheduled(cron = "0 0 1 * * ?") // Runs daily at 1 AM
    public void runInterestCalculation() {
        List<FixedDepositAccount> accounts =
fdAccountRepository.findActiveAccounts();
        accounts.forEach(this::applyInterest);
    }

    private void applyInterest(FixedDepositAccount fd) {
        double interest = interestCalculatorService.calculateInterest(fd);
        fd.addInterest(interest);
        fdAccountRepository.save(fd);
        transactionService.recordInterest(fd.getFdAccountNo(), interest);
    }
}
```

5.2 Service Layer – Interest Logic

```
java
public double calculateInterest(FixedDepositAccount fd) {
    if ("COMPOUND".equals(fd.getProduct().getInterestType())) {
        return compoundInterest(fd.getPrincipal(), fd.getRate(),
fd.getTermMonths());
    } else {
        return simpleInterest(fd.getPrincipal(), fd.getRate(),
fd.getTermMonths());
    }
}
```

6. Database Design – Interest Transactions

Table: fd_interest_transactions

Field	Type
transaction_id	INT (PK)
fd_account_no	VARCHAR(20)
interest_amount	DOUBLE
transaction_date	TIMESTAMP
status	VARCHAR(20)

7. Frontend (Optional)

- Admin dashboard to view upcoming interest payout schedules
- Trigger manual batch runs for testing

8. Output and Reporting

- Each FD account is updated with the new balance (if capitalized)
- A transaction record is generated for audit purposes
- Optional: Generate interest summary report (e.g., CSV)

Example output (console/logging):

```
plaintext
FD Account: FD000045
Principal: 100000
Interest this month: 550
New Principal (capitalized): 100550
```

9. Security and Operational Considerations

- **Idempotency:** Ensure batch job does not double-calculate if re-run
- **Access control:** Only `BANK_OFFICER` and `ADMIN` can trigger manual recalculations
- **Error handling:** Log accounts where calculation fails but proceed with others

Lab L15 – Pre-Maturity Withdrawal and Penalty Handling in Fixed Deposit Accounts

1. Objective

To implement and test the pre-maturity withdrawal process for Fixed Deposit (FD) accounts. This includes:

- Calculating interest accrued up to the withdrawal date.
- Applying product-specific penalties.
- Updating the FD account status and balance.
- Logging all withdrawal transactions for audit purposes.

2. Technical Overview

Fixed Deposits are normally withdrawn upon maturity. However, customers may request pre-mature withdrawals, subject to reduced interest or penalties.

Key technical aspects:

- Calculate actual interest earned till withdrawal date.
- Apply product-level penalty rules, such as reduced rate or withdrawal charges.
- Close the FD account and transfer the maturity proceeds to the customer's savings account or issue a payout.
- Ensure the operation is atomic: all steps must succeed, or none are applied.

This functionality is restricted to customers requesting withdrawal on their own account or by authorized bank officers.

3. System Workflow

```
plaintext
Customer/Bank Officer Initiates Withdrawal
↓
FD Module Checks Account Status and Term
↓
Calculates Accrued Interest till Today
```

↓
Applies Pre-Maturity Penalty Rules
↓
Updates FD Account Status to CLOSED
↓
Generates Withdrawal Transaction Record
↓
Transfers the Withdrawal Amount to Customer Account

4. Business Logic Explained

Business Rule	Example
Interest payable till withdrawal date	If withdrawn after 6 months on a 1-year FD, pay 6-month interest
Penalty on interest	Reduce accrued interest by 1% or as configured
Fixed withdrawal charge	₹200 as a flat withdrawal fee (optional)
Account status change	From ACTIVE → CLOSED
Statement generation	Pre-maturity closure reflected in FD summary

5. API Design

5.1 Endpoint

plaintext
POST /api/fd/account/withdraw

5.2 Request Payload

```
json
{
  "fdAccountNo": "FD123456",
  "withdrawalDate": "2025-11-15",
  "transferAccount": "SB000567"
}
```

5.3 Response

```
json
{
  "status": "success",
  "message": "FD account closed successfully.",
  "withdrawalAmount": 104500,
  "penaltyApplied": 500
}
```

6. Backend Implementation

6.1 Controller Layer

```
java
@PreAuthorize("hasRole('CUSTOMER') or hasRole('BANK_OFFICER')")
@PostMapping("/fd/account/withdraw")
public ResponseEntity<?> withdrawFD(@RequestBody WithdrawalRequest req,
Authentication auth) {
    String username = auth.getName();
    return
ResponseEntity.ok(fdWithdrawalService.processWithdrawal(req.getFdAccountNo(),
req.getWithdrawalDate(), req.getTransferAccount(), username));
}
```

6.2 Service Layer

```
java
public WithdrawalResponse processWithdrawal(String fdAccountNo, LocalDate
withdrawalDate, String transferAccount, String requestedBy) {
    FixedDepositAccount fd = fdAccountRepo.findById(fdAccountNo)
        .orElseThrow(() -> new
FDNotFoundException(fdAccountNo));

    validateWithdrawalEligibility(fd, withdrawalDate);
    double accruedInterest = calculateInterestTill(fd, withdrawalDate);
    double penalty = applyPenalty(fd.getProductCode(), accruedInterest);
    double payoutAmount = fd.getPrincipal() + accruedInterest - penalty;

    fd.setStatus("CLOSED");
    fdAccountRepo.save(fd);

    transactionService.recordWithdrawal(fdAccountNo, payoutAmount, penalty);
    paymentService.transferToAccount(transferAccount, payoutAmount);

    return new WithdrawalResponse("success", "FD account closed
successfully.", payoutAmount, penalty);
}
```

6.3 Penalty Application Logic

```
java
public double applyPenalty(String productCode, double interestEarned) {
    Product product = productService.getProductDetails(productCode);
    double penaltyRate = product.getPreMaturityPenaltyPercent(); // e.g.,
1.0 means 1%
    return (interestEarned * penaltyRate) / 100;
}
```

7. Database Changes

Table	Update
fd_accounts	Update status to CLOSED, update closed_at timestamp
fd_interest_transactions	Record final interest transaction
fd_transactions	Record withdrawal and penalty transactions

8. Security and Validation

- Only the FD **account owner** or a **BANK_OFFICER** can initiate the withdrawal.
- Account status must be ACTIVE; withdrawal is rejected if already closed.
- Withdrawal date must be $\geq$ creation date and $\leq$ maturity date.

9. Frontend Workflow (Angular)

- Display a **Withdraw FD** button only for active accounts.
- Confirm action before proceeding.
- Show final payout amount and penalty summary.

10. Expected Outcome

- FD account is closed successfully.
- Customer receives the correct payout amount (Principal + reduced interest – penalty).
- Withdrawal transaction recorded for audit.
- Error is shown if unauthorized user or invalid withdrawal request.

12. Sample SQL (Withdrawal Transaction)

```
sql
INSERT INTO fd_transactions (fd_account_no, txn_type, amount, txn_date,
remarks)
VALUES ('FD123456', 'WITHDRAWAL', 104500, NOW(), 'Premature withdrawal with
penalty applied');
```

Lab L16 – FD Account Creation with Initial Deposit Transaction Logging

1. Objective

To extend the Fixed Deposit (FD) account creation process by incorporating:

- Automated recording of the initial deposit transaction during account creation.
- Ensuring the transaction history is initialized properly for future auditing and reporting.
- Associating the FD account with a transaction lifecycle right from creation.

2. Technical Overview

In real-world banking systems, every financial operation is tracked as a **transaction**. When a new FD account is created, an initial **DEPOSIT transaction** is recorded automatically, establishing the opening balance and linking the account to the general ledger.

This lab focuses on:

- Inserting a **DEPOSIT transaction** for the FD creation event.
- Capturing audit metadata: who created the account, when, and for what amount.
- Maintaining consistency across the `fd_accounts` and `fd_transactions` tables.

3. Workflow

```
plaintext
[Angular UI] - Submit FD Account Creation Request
    ↓
[Spring Boot] - Validate Product and Customer
    ↓
Create FD Account Record in MySQL
    ↓
Generate Initial DEPOSIT Transaction
    ↓
Save Both Records in a Single Database Transaction
    ↓
Return Account Number and Initial Transaction ID
```

4. Database Schema (MySQL)

4.1 fd_accounts

Field	Type
fd_account_no	VARCHAR(20) PK
customer_id	VARCHAR(20)
product_code	VARCHAR(10)
principal	DOUBLE
interest_rate	DOUBLE
term_months	INT
status	VARCHAR(20)
created_at	TIMESTAMP

4.2 fd_transactions

Field	Type
txn_id	INT PK
fd_account_no	VARCHAR(20)
txn_type	VARCHAR(20)
amount	DOUBLE
txn_date	TIMESTAMP
remarks	VARCHAR(255)

5. Backend Implementation

5.1 Controller Layer

```
java
@PreAuthorize("hasRole('BANK_OFFICER')")
@PostMapping("/fd/account/create-with-txn")
public ResponseEntity<?> createFDWithTxn(@RequestBody FDAccountRequest
request, Authentication auth) {
    String username = auth.getName();
```

```

        return
        ResponseEntity.ok(fdAccountService.createFDAccountWithTransaction(request,
        username));
    }

```

5.2 Service Layer

```

java
@Transactional
public FDAccountCreationResponse
createFDAccountWithTransaction(FDAccountRequest req, String createdBy) {
    String fdAccountNo =
    fdAccountNumberGenerator.generate(req.getBranchCode());
    double rate = productService.fetchRateForProduct(req.getProductCode(),
    req.getTermMonths());

    // Create FD account
    FixedDepositAccount fd = new FixedDepositAccount(fdAccountNo,
    req.getCustomerId(),
    req.getProductCode(), req.getPrincipalAmount(), rate,
    req.getTermMonths());
    fd.setStatus("ACTIVE");
    fdAccountRepo.save(fd);

    // Record initial deposit transaction
    FDTransaction txn = new FDTransaction(fdAccountNo, "DEPOSIT",
    req.getPrincipalAmount(), "Initial Deposit");
    fdTransactionRepo.save(txn);

    return new FDAccountCreationResponse(fdAccountNo, txn.getTxnId(),
    "Account and initial deposit created.");
}

```

6. Example API Request and Response

6.1 API Request Payload

```

json
{
  "customerId": "CUST001",
  "productCode": "FDSTD",
  "principalAmount": 200000,
  "termMonths": 12,
  "branchCode": "BR001"
}

```

6.2 API Success Response

```

json
{
  "fdAccountNo": "BR0010001235",
  "initialTransactionId": 2001,
  "message": "Account and initial deposit created."
}

```

7. Security & Validation

- Only **BANK_OFFICER** role can create new FD accounts.
- Validate:
 - Product exists and is active
 - Term months fall within product-defined limits
 - Principal amount meets minimum deposit requirements

8. Frontend Integration (Angular)

- Enhance FD creation form to submit via `/fd/account/create-with-txn`
- On success:
 - Show FD Account No and Transaction ID
 - Provide a link to View Transaction History

9. Example SQL Insertions

```
sql
-- FD Account Creation
INSERT INTO fd_accounts (fd_account_no, customer_id, product_code, principal,
interest_rate, term_months, status, created_at)
VALUES ('BR0010001235', 'CUST001', 'FDSTD', 200000, 6.5, 12, 'ACTIVE',
NOW());

-- Initial Deposit Transaction
INSERT INTO fd_transactions (fd_account_no, txn_type, amount, txn_date,
remarks)
VALUES ('BR0010001235', 'DEPOSIT', 200000, NOW(), 'Initial Deposit');
```

10. Expected Outcome

- A new FD account is created.
- An initial DEPOSIT transaction is automatically recorded.
- FD Account and its transaction are linked through the `fd_account_no`.

Lab L17 – Batch Scheduler Implementation for Interest Calculation and Statement Generation

1. Objective

To implement a batch scheduler using Spring Boot that automates:

- Periodic interest calculation for all active Fixed Deposit accounts.
- Generation of account statements summarizing interest accrual and balances.

This lab introduces batch processing, enabling the system to process a large number of accounts automatically at scheduled intervals.

2. Technical Overview

Unlike individual API calls, batch processing:

- Runs automatically on a scheduled basis (daily, monthly).
- Iterates over all eligible FD accounts.
- Calculates accrued interest according to account and product configurations.
- Updates account balances or generates payable interest.
- Creates monthly statements reflecting interest and balance updates.

Batch jobs will run via Spring Boot's `@Scheduled` annotation, and their status should be logged.

3. System Workflow

```
plaintext
[Scheduler Trigger]
↓
Fetch all ACTIVE FD Accounts
↓
For each account:
↓
Calculate Interest for the Period
↓
Generate or Update Account Statement
↓
Record Interest Transaction
↓
Log Batch Completion
```

4. Batch Scheduler Implementation (Spring Boot)

4.1 Batch Scheduler Class

```
java
@Component
public class BatchScheduler {

    @Autowired
    private InterestCalculationService interestService;

    @Autowired
    private StatementService statementService;

    @Scheduled(cron = "0 0 3 * * ?") // Daily at 3 AM
    public void runDailyBatch() {
        interestService.calculateDailyInterest();
        statementService.generateDailyStatements();
    }
}
```

4.2 Example Cron Expressions

Schedule Type	Cron Example	Runs At
Daily	0 0 3 * * ?	3:00 AM every day
Monthly	0 0 2 1 * ?	2:00 AM on the 1st of month

5. Interest Calculation Service Example

```
java
public void calculateDailyInterest() {
    List<FixedDepositAccount> accounts =
        fdAccountRepo.findAllActiveAccounts();

    for (FixedDepositAccount fd : accounts) {
        double interest = calculateInterest(fd);
        fd.addAccruedInterest(interest);
        fdAccountRepo.save(fd);
        fdTransactionRepo.save(new FDTransaction(fd.getFdAccountNo(),
            "INTEREST", interest, "Daily Interest"));
    }
}
```

6. Statement Generation Service Example

```
java
public void generateDailyStatements() {
    List<FixedDepositAccount> accounts =
        fdAccountRepo.findAllActiveAccounts();

    for (FixedDepositAccount fd : accounts) {
        FDStatement statement = new FDStatement(fd.getFdAccountNo(),
            LocalDate.now(), fd.getPrincipal(), fd.getAccruedInterest());
        fdStatementRepo.save(statement);
    }
}
```

7. Database Tables

Table Name	Purpose
fd_interest_transactions	Logs each interest accrual event
fd_statements	Stores summary of FD balances and accrued interest

8. Logging and Monitoring

- Log batch start and completion with timestamp:

```
plaintext
Batch Interest Calculation started at 03:00 AM
Processed 150 accounts. Completed at 03:02 AM
```

- If any account fails (e.g., product mismatch), log error and continue.

9. Security & Error Handling

- Only the **Scheduler process** should execute these operations.
- Bank admin users may later be allowed to trigger on-demand batches (Lab 18).
- Failed account calculations should be logged for manual review.

10. Expected Output

- Active FD accounts accrue interest daily.
- Each FD account has a daily interest record in `fd_transactions`.
- Daily statement entries are available in `fd_statements`.

12. Example Batch Run Log

plaintext

```
[2025-11-01 03:00] Batch Start: Interest Calculation  
Processed Account: FD12345, Interest: 520  
Processed Account: FD12346, Interest: 480  
Total Accounts Processed: 150  
Batch Completed Successfully at 03:02 AM
```

Lab L18 – Batch Processing for Maturity Processing and Customer Notifications

1. Objective

To implement batch processing that:

- Identifies Fixed Deposit (FD) accounts reaching maturity.
- Calculates final maturity payouts.
- Closes matured FD accounts.
- Sends automated maturity notifications (email, WhatsApp) to customers.
- Records transactions in the general ledger and FD transaction logs.

This enables banks to manage maturity operations without manual intervention, ensuring timely payouts and customer alerts.

2. Technical Overview

The maturity process differs from daily interest calculation:

- It runs on or after the maturity date.
- Calculates final maturity amounts (principal + accrued interest).
- Marks the account as CLOSED.
- Triggers a payout and generates customer-facing notifications.
- Optionally generates operational reports for compliance and reconciliation.

The process is automated using Spring Boot's batch scheduler (@Scheduled).

3. Workflow

```
plaintext
Scheduled Maturity Batch Run
↓
Retrieve all ACTIVE FD accounts where maturity_date ≤ today
↓
Calculate maturity amount (Principal + Interest)
↓
Create MATURITY_PAYOUT transaction
↓
```

```
Close the FD Account (status = CLOSED)
↓
Send email/WhatsApp notification to the customer
↓
Log results for audit and compliance reporting
```

4. Maturity Calculation Logic

```
java
public double calculateMaturityAmount(FixedDepositAccount fd) {
    return fd.getPrincipal() + fd.getAccruedInterest();
}
```

5. Spring Boot Batch Implementation

5.1 Scheduler Class

```
java
@Component
public class MaturityBatchScheduler {

    @Autowired
    private MaturityProcessingService maturityService;

    @Scheduled(cron = "0 0 4 * * ?") // Runs daily at 4 AM
    public void runMaturityProcessing() {
        maturityService.processAllMaturedAccounts(LocalDate.now());
    }
}
```

5.2 Maturity Processing Service

```
java
public void processAllMaturedAccounts(LocalDate today) {
    List<FixedDepositAccount> maturedAccounts =
        fdAccountRepo.findMaturedAccounts(today);

    for (FixedDepositAccount fd : maturedAccounts) {
        double maturityAmount = calculateMaturityAmount(fd);

        fd.setStatus("CLOSED");
        fdAccountRepo.save(fd);

        fdTransactionRepo.save(new FDTransaction(
            fd.getFdAccountNo(),
            "MATURITY_PAYOUT",
            maturityAmount,
            "Maturity Payout on " + today
        ));

        notificationService.sendMaturityNotice(fd.getCustomerId(),
            maturityAmount, today);
    }
}
```

}

6. Database Impact

Table Name	Action
fd_accounts	Update status to CLOSED, save closure date
fd_transactions	Insert MATURITY_PAYOUT transaction
fd_interest_transactions	Finalize interest accrual
notifications_log	Record sent email/SMS/WhatsApp

7. Notification Example (Email/WhatsApp)

Subject: Your Fixed Deposit Has Matured Successfully

Dear [Customer Name],

Your Fixed Deposit [FD12345] has matured on [01-Dec-2025].

Maturity Amount: ₹105,000.00 has been processed.

Thank you for banking with us.

8. Logging and Monitoring

Batch logs example:

```
plaintext
[2025-12-01 04:00] Maturity Batch Start
Account FD12345 matured for ₹105,000
Account FD12346 matured for ₹75,500
Batch Completed: 2 Accounts Processed
```

Failures (e.g., account already closed) should be logged and skipped without stopping the batch.

9. Expected Output

- Matured FD accounts are closed automatically.
- Maturity payouts are recorded in `fd_transactions`.
- Customers receive notifications.
- Audit logs capture batch execution summary.

10. Security Considerations

- Batch job should not process accounts that are already CLOSED or withdrawn prematurely.
- Email/WhatsApp API keys and templates should be secured in environment variables.
- Sensitive customer information should not be logged in plain text.

Lab L19 – Maturity Calculation, Closure, and Final Payout Execution

1. Objective

To implement the final maturity processing workflow for FD accounts, including:

- Re-confirmation of maturity calculations,
- Closure of the FD account,
- Automated payout of the maturity amount to the customer's savings account,
- Logging of final closure transactions.

This lab ensures that matured accounts are fully processed and funds are transferred seamlessly.

2. Technical Overview

While Lab 18 handled the detection and preliminary maturity handling in a batch, Lab 19 focuses on the financial closure actions, simulating real-world banking processes such as:

- Final payout calculation,
- Account status update,
- GL entries for debit/credit,
- Fund transfer to customer savings account.

3. Maturity Closure Workflow

```
plaintext
User or Batch Initiates Maturity Closure
↓
Revalidate maturity date and interest calculations
↓
Calculate maturity payout = Principal + Final Interest
↓
Transfer payout to customer's savings account
↓
Mark FD Account as CLOSED
↓
Generate maturity transaction record
```

4. Backend Implementation

4.1 API Controller (Optional for Manual Closure)

```
java
@PreAuthorize("hasRole('BANK_OFFICER') or hasRole('ADMIN')")
@PostMapping("/fd/account/manual-close")
public ResponseEntity<?> closeFDManually(@RequestParam String fdAccountNo) {
    return
    ResponseEntity.ok(maturityService.closeMaturedAccount(fdAccountNo));
}
```

4.2 Service Layer Logic

```
java
public MaturityResponse closeMaturedAccount(String fdAccountNo) {
    FixedDepositAccount fd = fdAccountRepo.findById(fdAccountNo)
        .orElseThrow(() -> new RuntimeException("FD account not found"));

    if (!fd.isMatured()) {
        throw new IllegalStateException("FD not yet matured.");
    }

    double maturityAmount = fd.getPrincipal() + fd.getAccruedInterest();

    // Transfer maturity amount
    paymentService.transferToSavings(fd.getCustomerId(), maturityAmount);

    // Record maturity transaction
    fdTransactionRepo.save(new FDTransaction(fdAccountNo, "MATURITY_PAYOUT",
maturityAmount, "Final maturity payout"));

    fd.setStatus("CLOSED");
    fdAccountRepo.save(fd);

    return new MaturityResponse("CLOSED", maturityAmount, LocalDate.now());
}
```

5. Transaction and Ledger Updates

Transaction Type	Ledger Action
MATURITY_PAYOUT	Debit: FD Liabilities GL
	Credit: Customer Savings

6. Example SQL for Transaction

```
sql
INSERT INTO fd_transactions (fd_account_no, txn_type, amount, txn_date,
remarks)
```

```
VALUES ('FD123456', 'MATURITY_PAYOUT', 105000, NOW(), 'Final maturity  
payout');
```

7. Fund Transfer

A simulated transfer:

```
java  
public void transferToSavings(String customerId, double amount) {  
    savingsAccountService.creditAccount(customerId, amount);  
}
```

In real-world systems, this would integrate with a Core Banking System (CBS) or a payment gateway.

8. Expected Outcome

- The FD account is closed.
- The maturity amount is transferred to the savings account.
- All transactions are logged.
- The final account statement reflects the maturity payout.

9. Logging Example

```
plaintext  
FD Account FD123456 matured. ₹105,000 transferred to savings account  
SB000789.  
Account status updated to CLOSED.
```

10. Security and Compliance

- Only BANK_OFFICER or ADMIN roles can trigger manual closure.
- No duplicate maturity processing allowed (check status before closure).
- All financial actions logged for audit compliance.

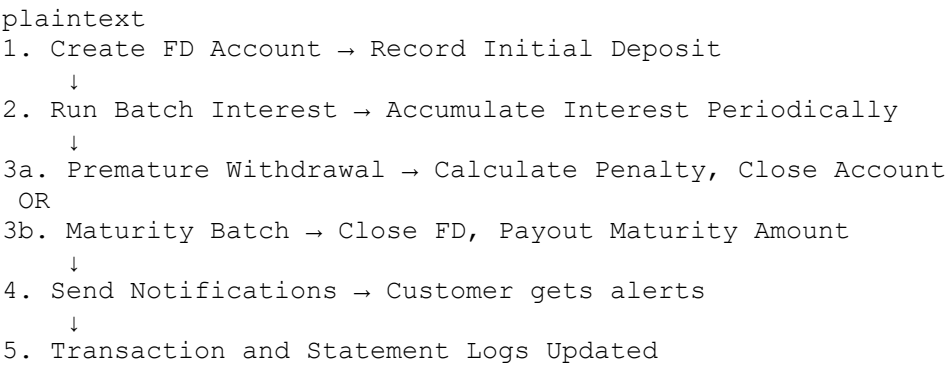
Lab L20 – Full FD Module Workflow Integration Testing

1. Objective

To execute a complete end-to-end functional test of the FD Module, validating the full lifecycle of an FD account from:

- Account creation,
- Periodic interest calculation (batch),
- Premature withdrawal or maturity closure,
- Fund transfer and transaction logging,
- Reporting and user access control.

2. Technical Workflow Overview



3. End-to-End Testing Flow

Step	Action	Method	Role
1	Create FD account	POST /fd/account/create-with-txn	BANK_OFFICER
2	Validate initial deposit recorded	GET /fd/account/{id}	CUSTOMER
3	Run daily interest batch	Scheduled / Admin Trigger	SYSTEM / ADMIN
4	Check interest transactions	GET /fd/transactions	CUSTOMER

Step	Action	Method	Role
5	(Optional) Withdraw before maturity	POST /fd/account/withdraw	CUSTOMER
6	Run maturity batch (post term end)	Scheduled	SYSTEM
7	Verify account closed and maturity payout posted	GET /fd/account/{id}	CUSTOMER
8	Verify customer received payout	GET /savings/account/{id}	CUSTOMER
9	Generate FD Summary Report	GET /report/fd-summary	ADMIN

4. Tools for Integration Testing

Layer	Tool
API	Postman / Swagger
Database	MySQL Workbench
Frontend	Angular App
Batch Logs	Spring Boot Logs
Notification	Mock SMTP / WhatsApp Sandbox APIs

5. Testing Scenarios

Scenario 1 – Successful FD Lifecycle

- Account Created → Interest Credited Daily → Closed on Maturity → Funds transferred to savings account

Scenario 2 – Premature Withdrawal

- Account Created → Withdrawn at 6 months → Penalty applied → Remaining funds transferred

Scenario 3 – Failed Withdrawal Attempt

- Attempt withdrawal on CLOSED account → Returns error → Account status remains unchanged

Scenario 4 – Notification Validation

- Verify that creation, interest payout, and maturity events trigger notifications.

6. Expected System Outputs

- FD account and transaction tables show complete life-cycle data.
- Notifications and statements are correctly generated.
- No unauthorized actions are allowed.

7. User Role Validation

Role	Allowed Actions
CUSTOMER	View account, withdraw FD, view statements
BANK_OFFICER	Create FD, close FD, view all accounts
ADMIN	Run batch processes, generate reports

8. Example Log Summary

```
plaintext
[2025-12-02 01:00] FD Account BR0010001235 created for CUST123
[2025-12-10 03:00] Daily Interest ₹540 credited to BR0010001235
[2026-01-15 04:00] Premature withdrawal for FD BR0010001235, payout ₹103,250
processes
```

Lab L21 – Reporting Module API Development and Secure Integration

1. Objective

To design and implement the Reporting Module that provides secure access to various operational and compliance reports, including:

- FD Account Status Summary
- Interest Transaction History
- Maturity Summary
- Customer Portfolio

This lab also integrates the reporting module with the authentication and authorization system, ensuring that reports are securely accessed only by authorized users.

2. Technical Overview

Key features of the Reporting Module:

- Fetch aggregated data from multiple modules (`fd_accounts`, `fd_transactions`, `products`).
- Provide REST APIs to serve report data in **JSON or CSV** formats.
- Secure API endpoints so:
 - **CUSTOMER** → can only view their own FD portfolio.
 - **BANK_OFFICER / ADMIN** → can view summary and all customer data.
- Optionally generate and export reports in CSV/PDF using libraries like Apache POI or iText.

3. System Workflow

```
plaintext
Request: /api/report/fd-summary
↓
Validate JWT Token & Role
↓
Execute SQL Queries / Aggregate from Services
```

↓
Format Data as JSON/CSV
↓
Return to API Caller or Generate Downloadable File

4. API Specifications

4.1 FD Summary Report (Admin/Officer)

```
java
@PreAuthorize("hasRole('BANK_OFFICER') or hasRole('ADMIN')")
@GetMapping("/report/fd-summary")
public ResponseEntity<List<FDReportDto>> getFDSummary() {
    return ResponseEntity.ok(reportService.getFDSummary());
}
```

Sample Response:

```
json
[
  {
    "productCode": "FD_STD",
    "totalAccounts": 150,
    "totalPrincipal": 15000000,
    "totalInterestAccrued": 425000
  }
]
```

4.2 Customer Portfolio Report (Customer)

```
java
@PreAuthorize("hasRole('CUSTOMER')")
@GetMapping("/report/customer-portfolio")
public ResponseEntity<List<FDPortfolioDto>>
getCustomerPortfolio(Authentication auth) {
    String username = auth.getName();
    return
ResponseEntity.ok(reportService.getPortfolioForCustomer(username));
}
```

5. Database Aggregation Queries (Example)

FD Summary Query:

```
sql
SELECT product_code,
       COUNT(fd_account_no) AS totalAccounts,
       SUM(principal) AS totalPrincipal,
       SUM(accrued_interest) AS totalInterestAccrued
FROM fd_accounts
WHERE status = 'ACTIVE'
GROUP BY product_code;
```

6. Reporting Formats Supported

Format	Use Case
JSON	UI integration
CSV	Admin downloads
PDF	Compliance reports (optional, future scope)

7. Security Considerations

API	Accessible by
/report/fd-summary	ADMIN, BANK_OFFICER
/report/customer-portfolio	CUSTOMER (self only)
Unauthorized access	Returns 403 Forbidden

8. Expected Output

- Officers/Admins can view FD account statistics across all customers.
- Customers can only view their own FD portfolio.
- Generated reports match data from FD and transaction modules.

Lab L22 – Overall Solution Documentation and Test Plan Creation

1. Objective

To consolidate all technical work into a comprehensive solution document, prepare a Test Plan, and finalize API and system documentation.

2. Deliverables

- Solution Architecture Diagram
- Module-wise APIs (Swagger/Postman collections)
- ER Diagrams of all database entities
- Deployment diagram (Cloud infrastructure)
- Test Plan covering:
 - Unit Tests
 - Integration Tests
 - End-to-End Workflows
 - Security Tests
 - Batch Job Validation
- API versioning and backward compatibility notes

3. Example Test Cases

Test Case	Expected Result
Create FD account	FD created with correct principal and term
Unauthorized API access	403 Forbidden
Maturity batch before maturity date	No accounts processed
Pre-maturity withdrawal	Correct penalty applied

4. Tools

Tool	Purpose
Postman	API testing
JUnit/Mockito	Unit testing (Spring Boot)
Swagger/OpenAPI	API documentation
MySQL Workbench	Schema visualization

Lab L23 – Deployment on Azure App Service

1. Objective

To deploy the full application on Azure App Service / Kubernetes, ensuring it is cloud-hosted, accessible over HTTPS, and supports scaling.

2. Deployment Steps

Step	Tool / Service
Build Docker images for each module	Docker
Push to Azure Container Registry	ACR
Deploy to Azure App Service	Azure Portal
Configure environment variables	Azure Portal
Setup database in Azure SQL	Azure SQL DB
Configure scaling rules	Azure App Service

3. Cloud Setup Components

Component	Technology
Compute	Azure App Service / AKS
Database	Azure SQL Database
API Gateway (optional)	Azure API Management
Notifications	SendGrid, WhatsApp Cloud API
Monitoring	Azure Monitor / Logs

4. Deployment Verification

- Access application via **<https://<app-name>.azurewebsites.net/>**
- Verify all REST APIs
- Test database connectivity
- Trigger batch processes using Azure CLI/portal

Lab L24 – Final System Execution, Testing, and Troubleshooting

1. Objective

To execute the full banking application on cloud, validate all workflows, and resolve deployment/runtime issues.

2. Execution Activities

Activity	Verification
Create and close FD accounts	API / UI
Run batch jobs manually	Azure CLI
Validate data flow between modules	Database
Check error handling	Logs
Ensure secure access (JWT)	Postman

3. Troubleshooting Scenarios

Problem	Possible Cause	Resolution
API not reachable	Incorrect env variables	Update App Service
Batch job missed	Cron syntax error	Fix scheduler
Database connection fails	Network firewall	Update Azure SQL firewall
JWT validation fails	Expired tokens	Refresh tokens

Lab L25 – Summary Presentation and Future Enhancements

1. Objective

To prepare and deliver the final presentation covering:

- Project architecture
- Technologies used
- Demonstration of module interactions
- API walkthrough
- Cloud deployment overview
- Key business rules implemented

2. Recommended Presentation Structure

Slide No.	Topic
1	Project Overview
2	Problem Statement
3	System Architecture
4	Module-Wise Functionalities
5	Technology Stack
6	Cloud Deployment
7	End-to-End Demo
8	Key Learnings
9	Future Work
10	Q&A

3. Future Enhancements Suggestions

Feature	Description
Full CBS integration	Integrate with a real Core Banking System
Multi-language support	UI and API translations
Advanced fraud detection	Analyze unusual account behavior
Microservice decomposition	Break modules into individual services
Automated DevOps pipeline	CI/CD on Azure Pipelines

4. Expected Outcome

A fully deployed, tested, and demonstrated banking application prototype, ready for further productization.